SOFTWARE DESIGN SPECIFICATION

CARDIOCARE HEALTH ANALYZER

Prepared by

Sahana N B

USN: 4JK25MC079

Project Guide

Mrs. Amitha
Roshan Vakil

Course: MCA

1. SYSTEM DESIGN

1.1 Architectural Design

CardioCare Health Analyzer is a web-based application developed to help users analyse ECG images and access basic healthcare services. The system combines machine learning with a web application so that users can upload ECG images, view predicted categories, explore doctors and hospitals, and manage appointments.

The application is developed using Python and Flask. SQLite is used to store application data, while PyTorch is used to run the machine learning models. The system has three main types of users: general users or patients, doctors, and administrators.

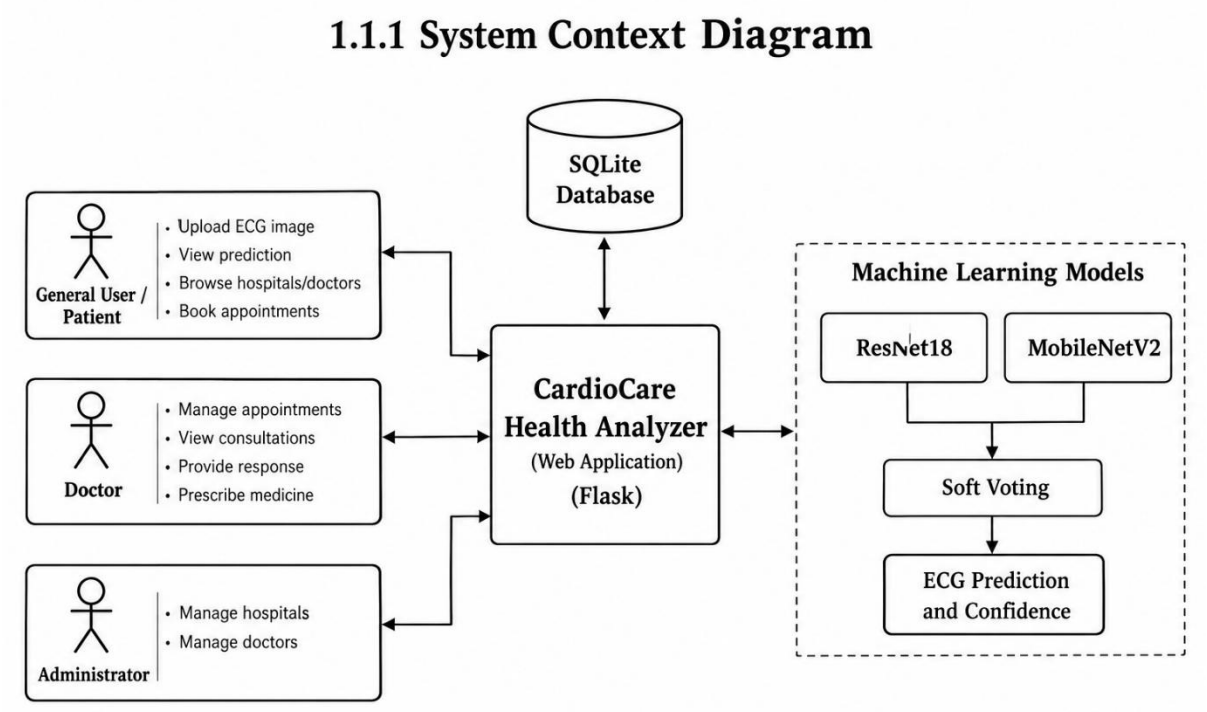
When a user uploads an ECG image, the application processes it and passes it to two trained machine learning models, ResNet18 and MobileNetV2. Their prediction probabilities are combined using soft voting to generate the final result. The application then displays the predicted ECG category and its confidence score.

The overall design connects the website, database and machine learning models so that the different features work together as one system.

1.1.1 System Context Diagram

The system context diagram shows how different users interact with CardioCare Health Analyzer and how the application connects to its database and machine learning models.

Diagram:

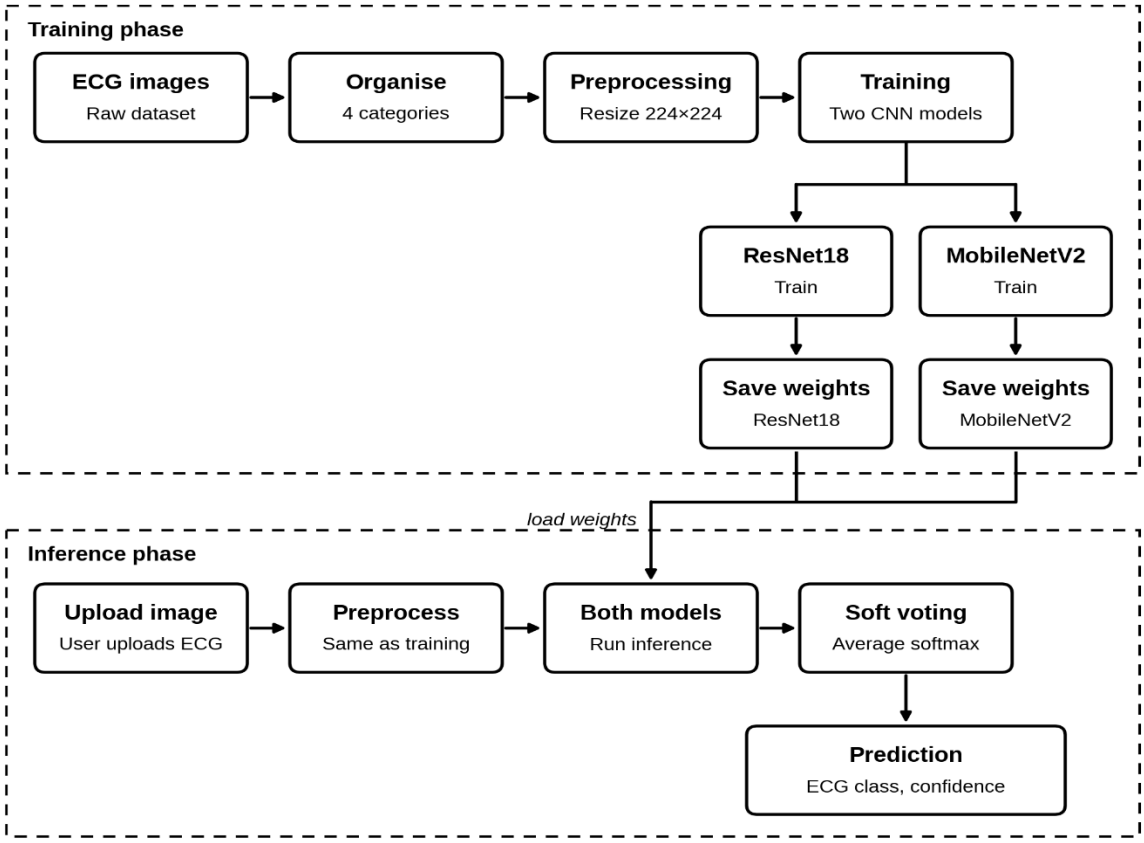


1.1.2 End-to-End ML Pipeline

The machine learning pipeline explains how ECG images are prepared, how the models are trained, and how the application generates predictions.

The project uses a dataset containing ECG images organised into four categories. The images are preprocessed before being used to train the models. Once training is completed, the model weights are saved and loaded by the Flask application whenever an ECG prediction is requested.

Diagram



2. DETAILED DESIGN

2.1 Dataset Design

The dataset is an important part of CardioCare Health Analyzer because the machine learning models learn from the ECG images provided during training. Each image is associated with a category that indicates the type of ECG represented.

The project uses separate training and testing folders. The training images help the models learn patterns, while the testing images are used to evaluate their performance on separate data.

2.1.1 Dataset Provenance and Permissions

The project uses an ECG image dataset for academic development and testing. The images are organised into training and testing folders, with separate folders for each category.

The previously recorded dataset size is 928 training images and 448 testing images. These counts should be checked against the final dataset before submitting the report.

The original dataset source, dataset owner, licence or permission details, and collection period should be added using the information provided by the original dataset publisher. These details should not be assumed if they have not been verified.

Since ECG images may contain health-related information, the dataset should be handled responsibly. Any personal information associated with the images should be protected, and the data should only be used according to its applicable permissions.

2.1.2 Dataset Schema and Data Dictionary

The dataset contains ECG image files and their corresponding class labels. The class labels are identified using the folders in which the images are stored.

Field	Type	Description
ECG Image	Image file	Contains the ECG image used for classification.
Class Folder	Text	Identifies the category to which the image belongs.
Class Label	Integer	Represents the category as a numerical value during model training.
Dataset Partition	Text	Identifies whether the image belongs to the training or testing set.
Image Dimensions	Numeric	Represents the original width and height of the image.
Image Path	File path	Identifies the location of the image file.

The images are processed before being given to the models. The folder names provide the labels required for supervised learning.

The class order must remain consistent during training and prediction. Otherwise, the application could display the wrong category name for a model's output.

2.1.3 Target / Label Definition

The target of the machine learning model is to classify an ECG image into one of four categories.

The four categories used in the project are:

1. ECG Images of Myocardial Infarction Patients

2. ECG Images of Patient that have History of MI
3. ECG Images of Patient that have abnormal heartbeat
4. Normal Person ECG Images

During training, the model learns patterns from images associated with these labels. When a new ECG image is uploaded, the trained model estimates which category best matches the image.

The model uses the labels provided by the dataset. It does not independently verify the medical correctness of those labels. Therefore, its output should be treated as a machine learning prediction rather than a confirmed diagnosis.

2.1.4 Partitioning and Leakage Prevention

The dataset is divided into training and testing sets to help evaluate the model fairly.

The training set is used to teach the models to recognise patterns in ECG images. The testing set is used to check how well the trained models perform on images that were not used for training. The improved ResNet18 training script also creates a validation split from the training data.

Care must be taken to prevent the same image, or a duplicate of it, from appearing in both the training and testing sets. If duplicate images appear in both sets, the model's performance may seem better than it really is.

Keeping the testing set separate from training helps provide a more reliable understanding of how the model performs on new images.

2.2 Preprocessing and Feature Engineering

Preprocessing prepares the ECG images so that they can be processed consistently by the machine learning models. The quality and consistency of this step can affect the prediction results.

2.2.1 Validation and Cleaning

Before an ECG image is processed, the application should check whether a file has been uploaded and whether it can be opened successfully.

Images that are missing, corrupted or unsupported should not be passed to the models. Instead, the application should display an appropriate error message.

The dataset should also be checked for unreadable images, incorrect class folders and duplicate images where possible.

The main preprocessing steps are:

- Check whether an image has been uploaded.
- Confirm that the image can be opened.

- Resize the image to 224 × 224 pixels.
- Convert the image into a tensor.
- Normalize the image using the selected mean and standard deviation values.

These steps help prepare the image in the format expected by the trained models.

2.2.2 Feature Selection / Extraction

CardioCare Health Analyzer uses deep learning to identify useful patterns directly from ECG images. It does not depend on manually selecting individual ECG features before classification.

The images are resized to 224 × 224 pixels and converted into tensors. Normalization is applied using the following values:

Mean: [0.485, 0.456, 0.406]

Standard deviation: [0.229, 0.224, 0.225]

ResNet18 and MobileNetV2 automatically learn visual features from the training images. Their convolutional layers process image patterns, and their classification layers use the learned features to estimate the ECG category.

The same preprocessing steps must be used during prediction to maintain consistency with the training process.

2.3 Model Design

The machine learning component uses two trained neural network architectures to classify ECG images. Their outputs are combined to generate the final prediction.

2.3.1 Baseline and Candidate Models

ResNet18

ResNet18 is a convolutional neural network used for image classification. It uses residual connections to help information flow through the network. In this project, the final classification layer is configured to produce outputs for the four ECG categories.

The trained model is saved as ecg_model_improved.pth.

MobileNetV2

MobileNetV2 is a lightweight convolutional neural network designed for efficient image processing. In the project's MobileNetV2 training script, the feature extractor is frozen and the final classification layer is trained for the four ECG categories.

The trained model is saved as mobilenet_model.pth.

Reason for using both models

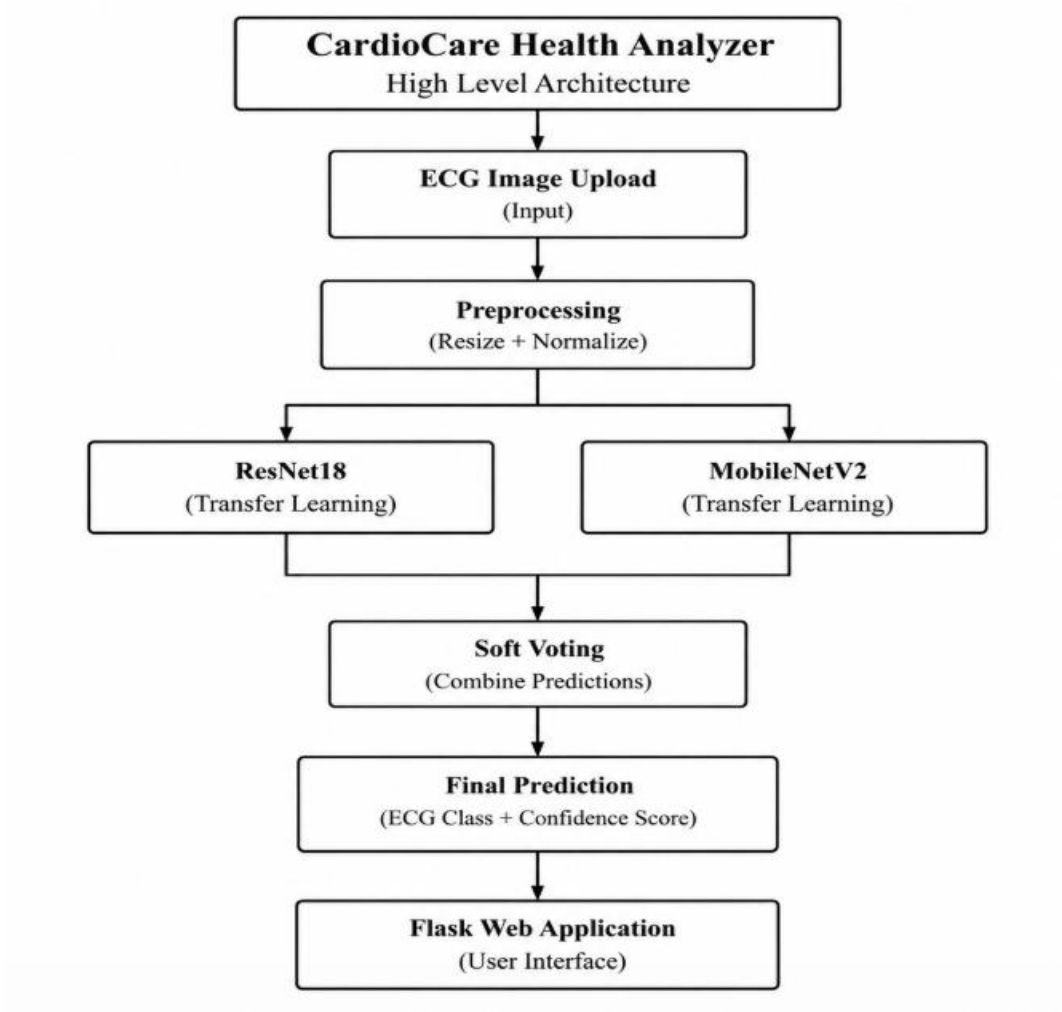
The application uses both ResNet18 and MobileNetV2 rather than relying on a single model. Each model produces probabilities for the four categories, and the application combines these probabilities using soft voting.

This approach allows both models to contribute to the final prediction. However, the combined model's performance must be verified through proper testing; using two models does not automatically guarantee higher accuracy.

2.3.2 Model Architecture

The model architecture describes how an ECG image is processed to produce a category prediction.

Diagram:



The input is a preprocessed ECG image. Both models independently generate probability values for the four categories. The application averages their probability outputs and selects the category with the highest combined probability.

The confidence score represents the model's estimated probability for the selected category. It should not be interpreted as a guarantee that the prediction is medically correct.

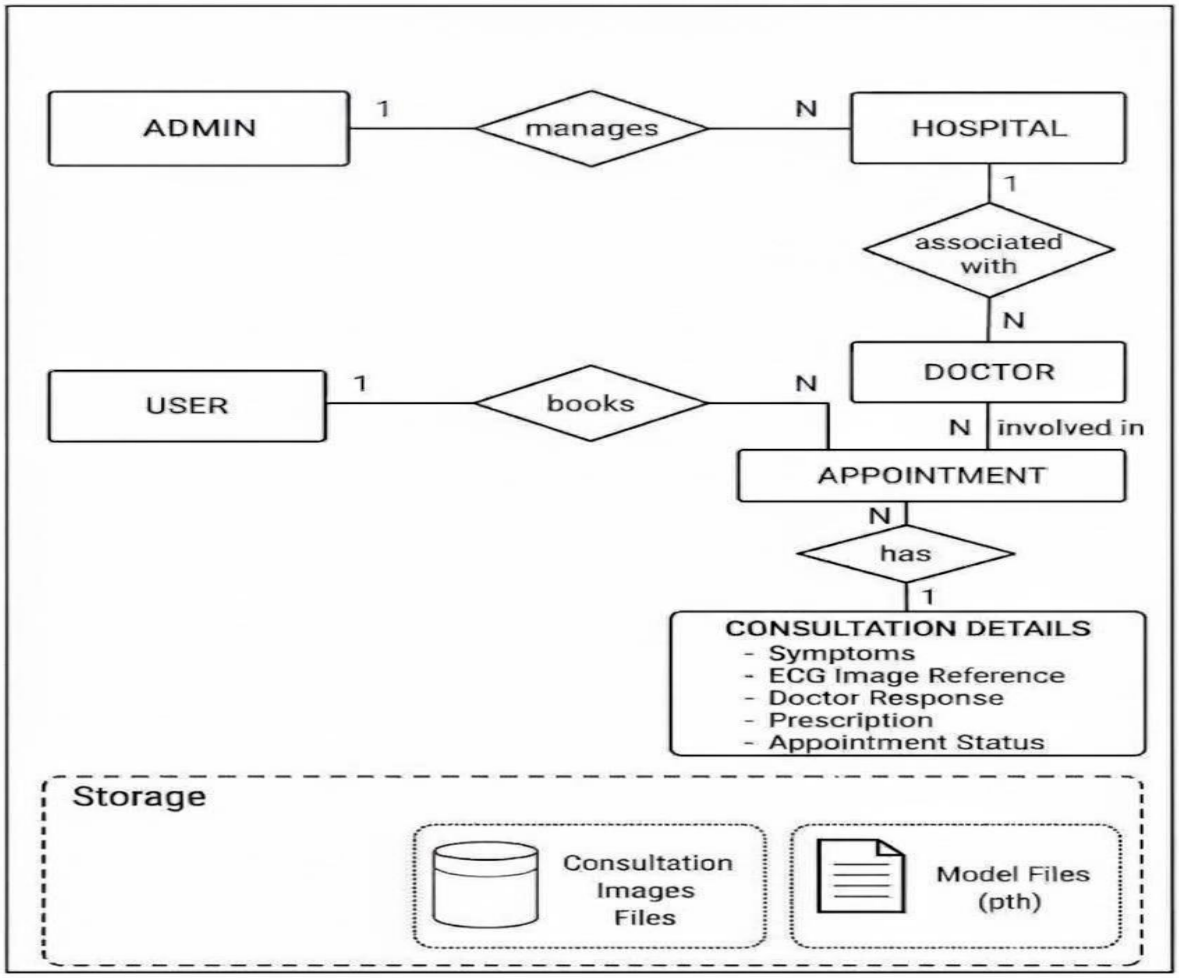
2.4 Database Design

The database stores the information needed for the website's normal operation. CardioCare Health Analyzer uses SQLite, with the database file named cardiocare.db.

The main record groups include users, administrators, hospitals, doctors and appointments. ECG images and consultation images are handled through file storage, with the application potentially storing their paths in the database.

2.4.1 ER Diagram / Storage Architecture

The following diagram shows the main conceptual relationships between the application's data groups.



The diagram represents the main relationships conceptually. The exact relationships and fields must be checked against the actual database schema before final submission.

2.4.2 Schema

The main database tables and storage components are described below.

Table or Storage	Purpose
users	Stores general user account information.
admins	Stores administrator account information.
hospitals	Stores hospital information.
doctors	Stores doctor profiles and professional information.
appointments	Stores appointment and consultation-related records.
Consultation image files	Stores images submitted during consultation, including files saved under static/consultation_ecgs.
Model files	Stores the trained model weights used for prediction.

SQLite manages the application's records, while the uploaded images and trained model files are stored separately.

The final report should include the actual column names, data types, primary keys and foreign keys after checking the project's database schema.

2.5 UML and Behavioral Design

This section explains how users interact with the application and how the system responds to their actions.

2.5.1 Use Case Diagram

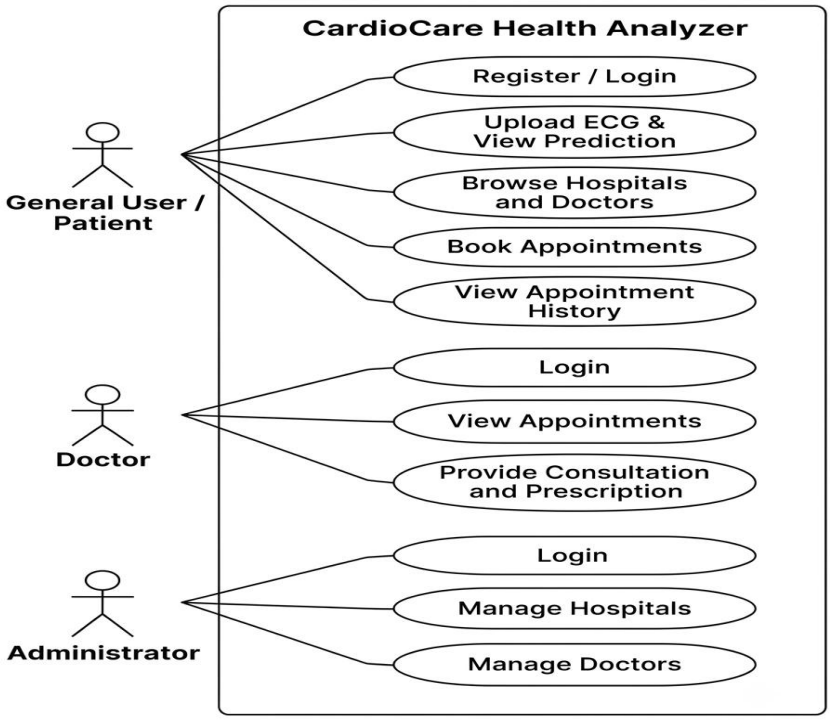
The use-case diagram identifies the main activities performed by each type of user.

General User / Patient: Registers or logs in, uploads ECG images, views prediction results, browses hospitals and doctors, books appointments, submits consultation details and views appointment or prescription information.

Doctor: Logs in, views appointments, accepts or rejects appointment requests, reviews submitted information and provides consultation responses and prescriptions.

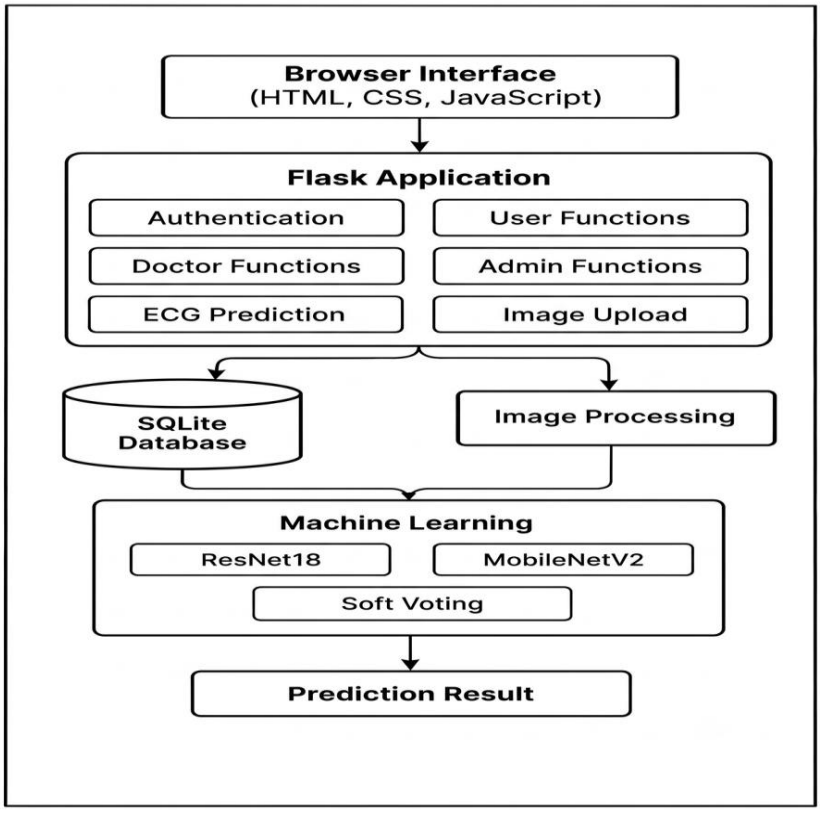
Administrator: Logs in and manages hospital and doctor information.

The system provides different functions according to the user's role, helping keep each user's activities organised.



2.5.2 Class / Component Diagram

The application is mainly organised around Flask routes, HTML templates, database operations and machine learning components.

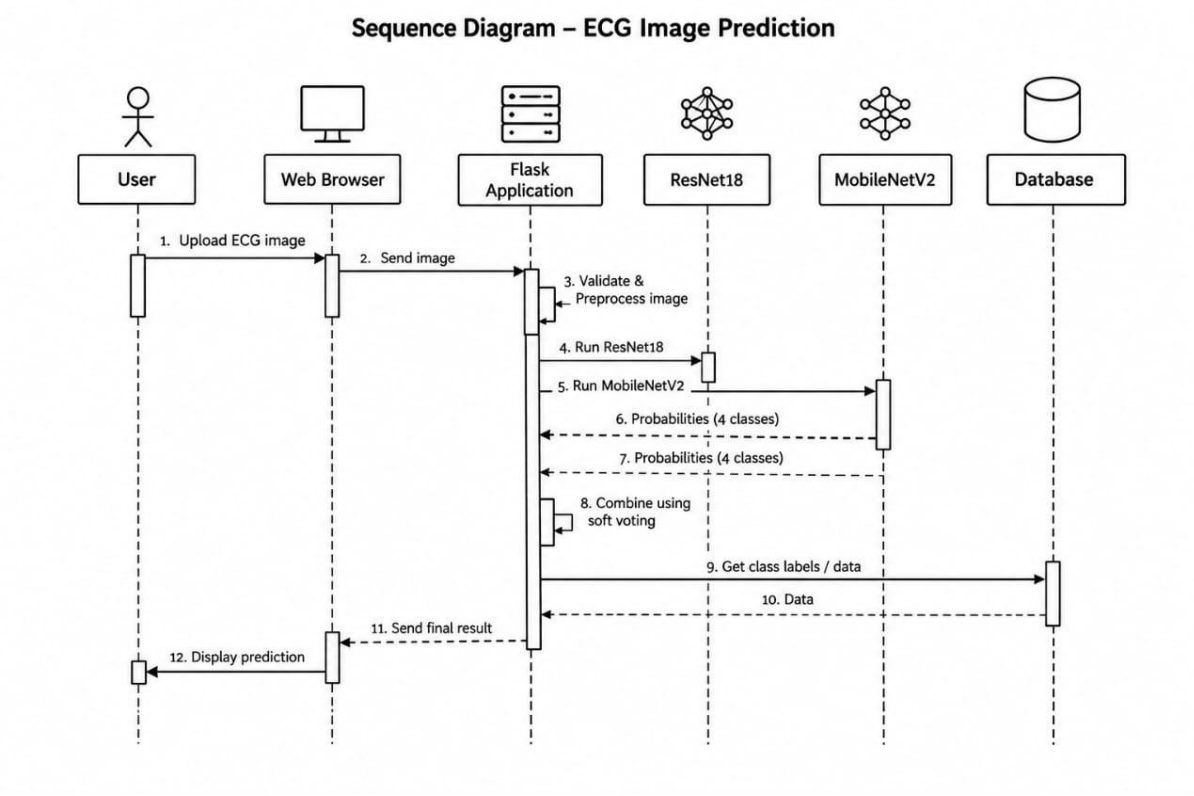


The browser provides the interface, and Flask processes the requests. The database stores application information, while the image-processing and machine learning components work together to generate ECG predictions.

This diagram represents the main software components and their dependencies rather than separate classes that may not exist in the source code.

2.5.3 Sequence Diagram

The sequence diagram explains the order of operations when a user uploads an ECG image.

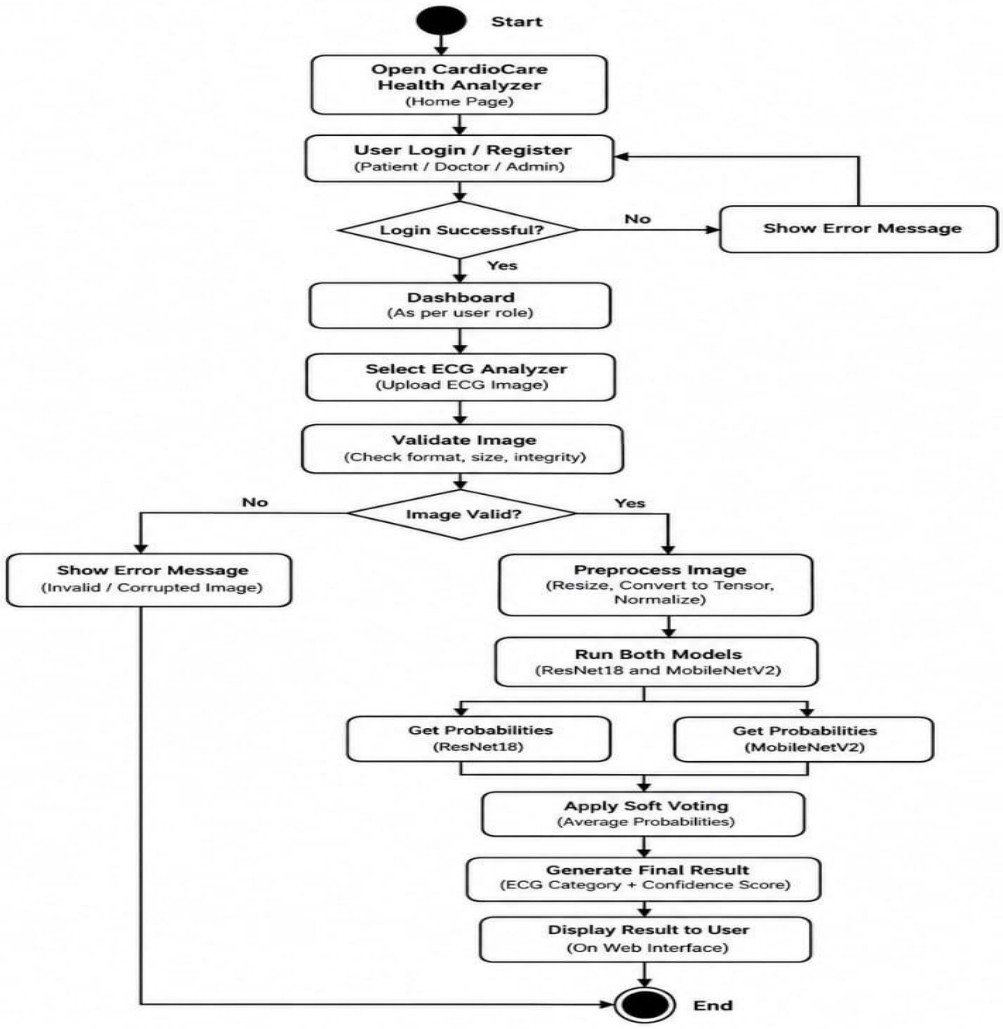


When the user uploads an ECG image, the browser sends it to Flask. Flask validates and preprocesses the image before passing it to the two machine learning models.

The models generate their probability outputs, and Flask combines them using soft voting. The final prediction is returned to the browser and displayed to the user.

2.5.4 Activity Diagram

The activity diagram shows the steps followed during ECG prediction, including what happens when an invalid image is submitted.



The process starts when the user opens the ECG analyzer and uploads an image. The application checks whether the image is valid.

If the image cannot be processed, an error message is displayed. Otherwise, the image is preprocessed and passed to both models. Their outputs are combined using soft voting, and the final category and confidence score are displayed.

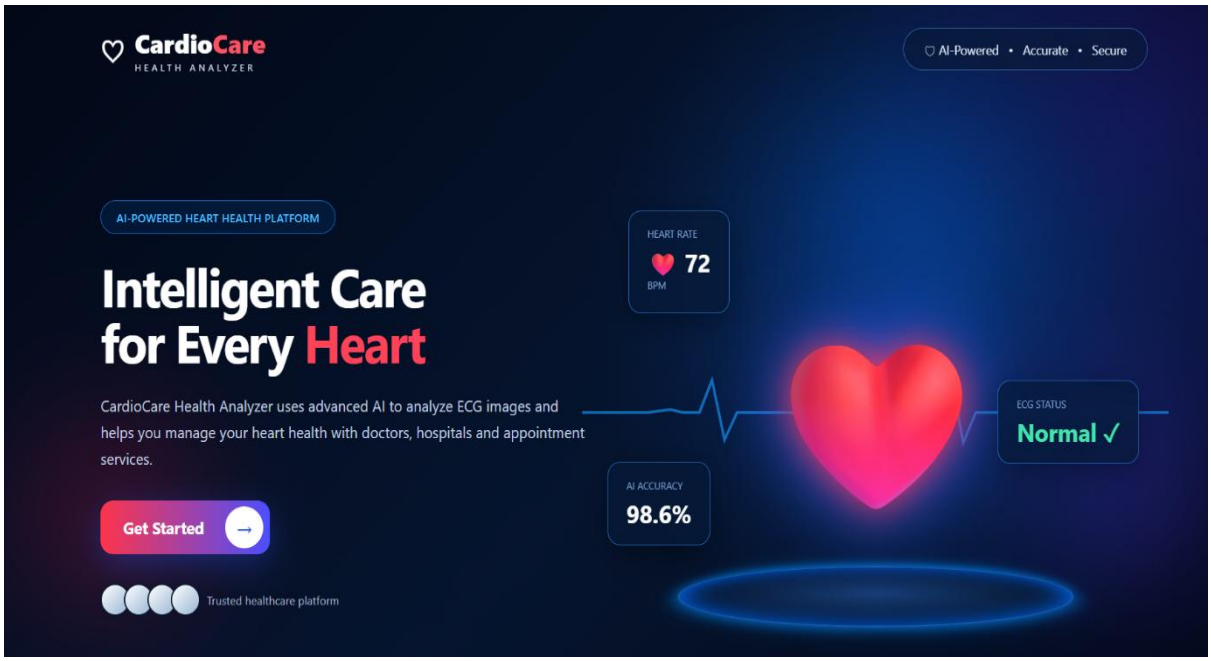
2.6 User Interface Design

The user interface is designed to make the application's features easy to understand and use. Different pages are provided for general users, doctors and administrators.

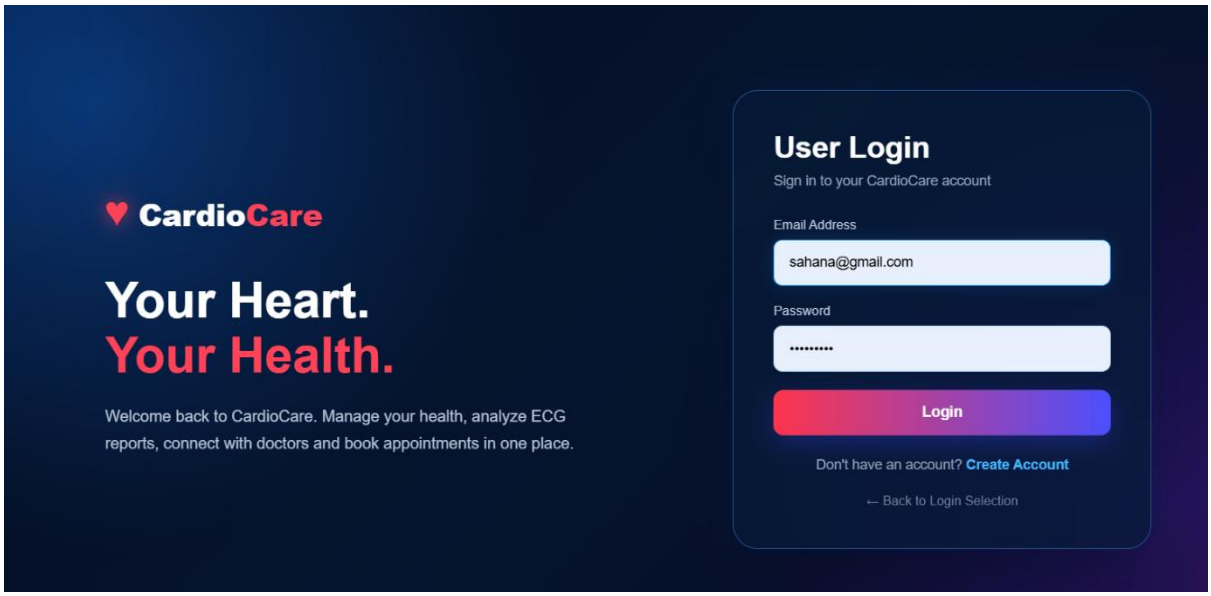
The main user interface includes the following pages:

1. Home page

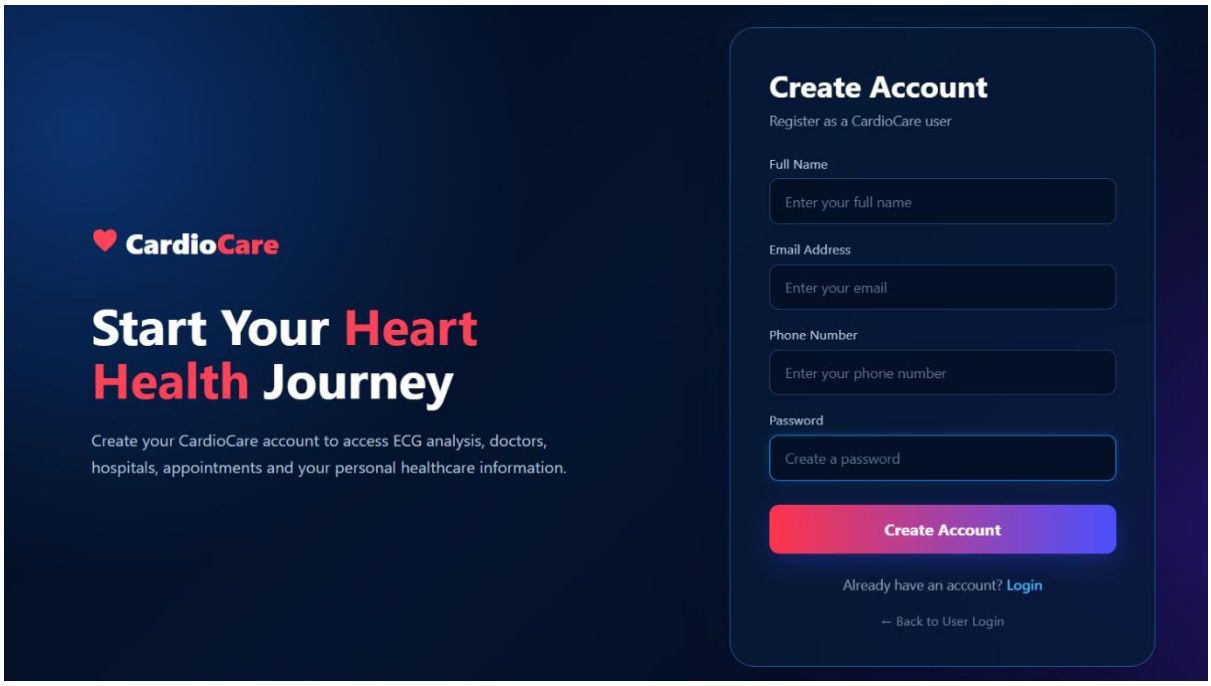
The home page provides access to the application.



2.Login page: Users can log in, and the system directs them to the relevant section based on their role.

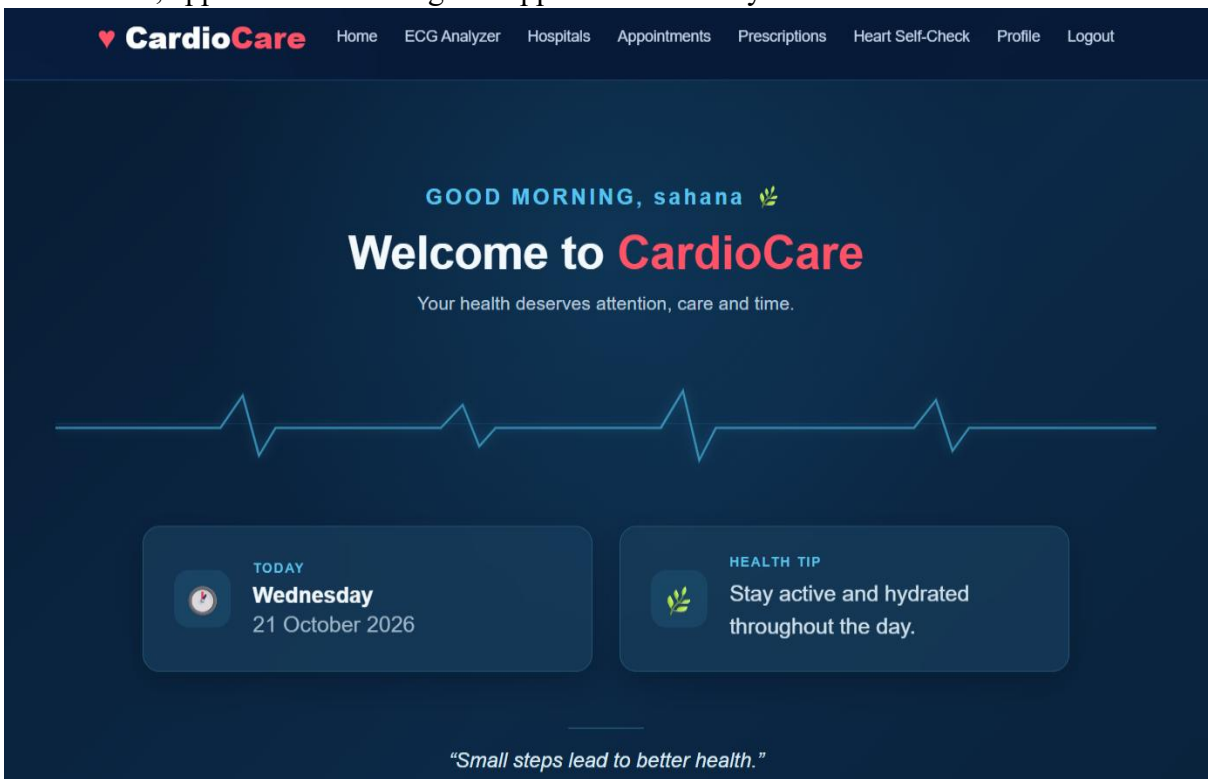


3.Registration page: Users can register and the system directs them to the relevant section based on their role.



4.User Dashboard

The dashboard provides navigation to the ECG Health Analyzer, hospital information, doctor information, appointment booking and appointment history.



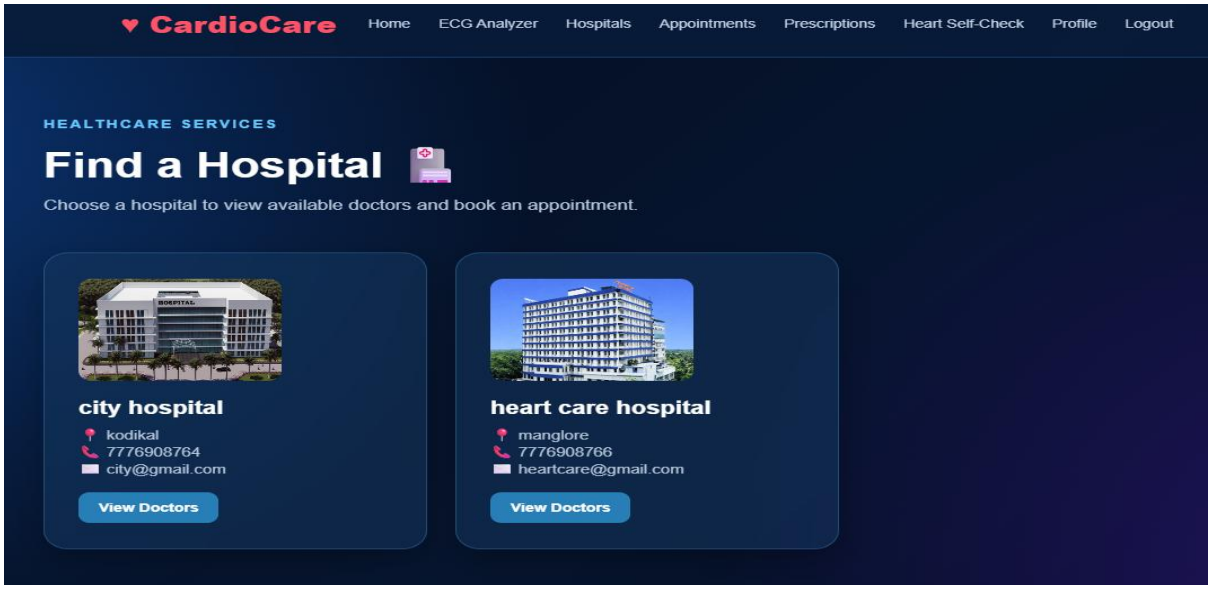
5.ECG Health Analyzer Page

This page allows users to upload an ECG image and view the predicted category and confidence score after processing.

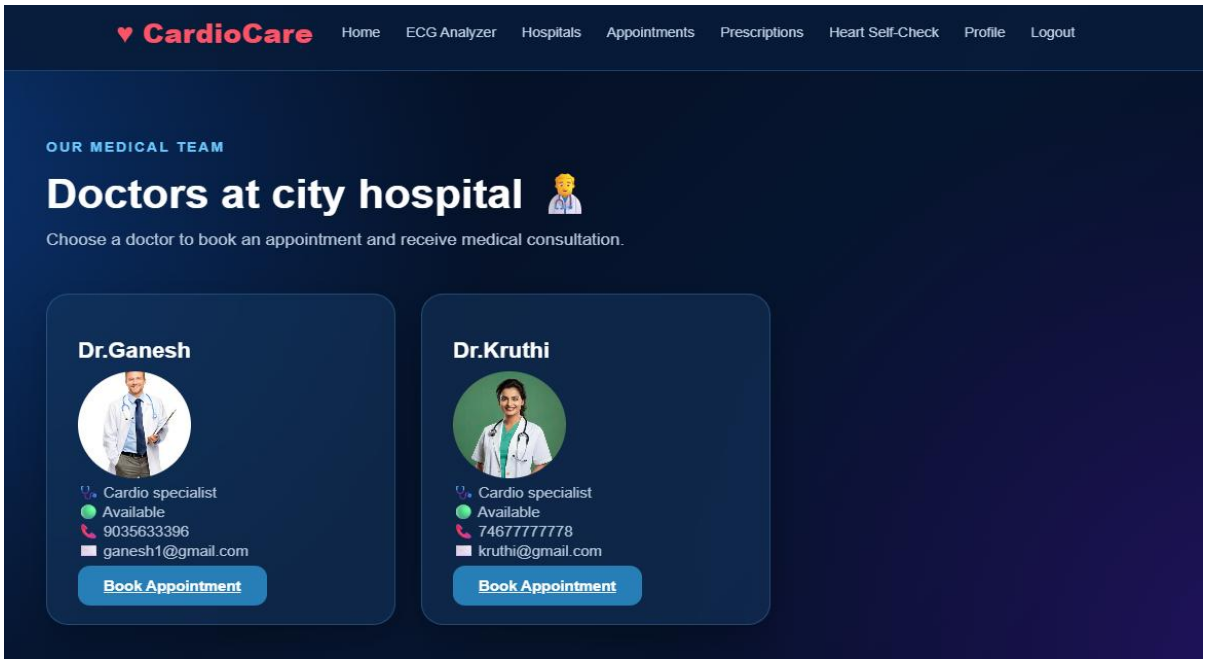


6. Hospital page

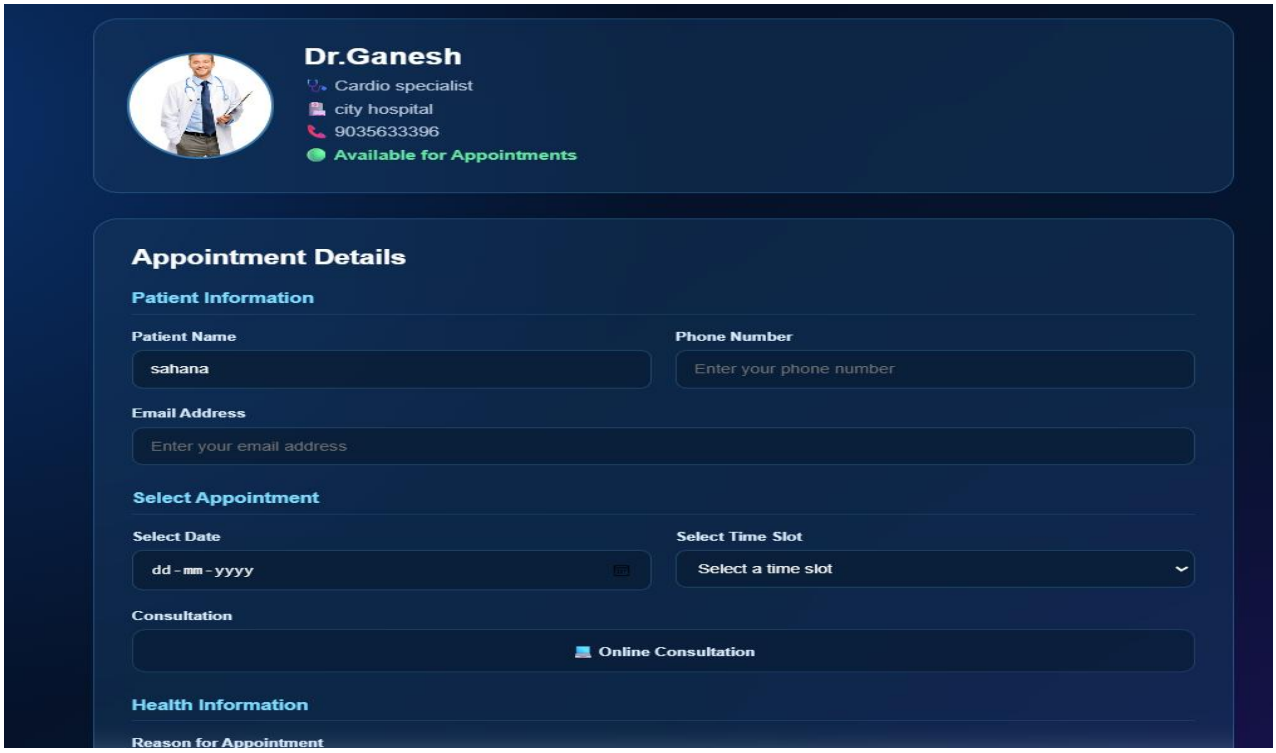
These pages display hospital information.



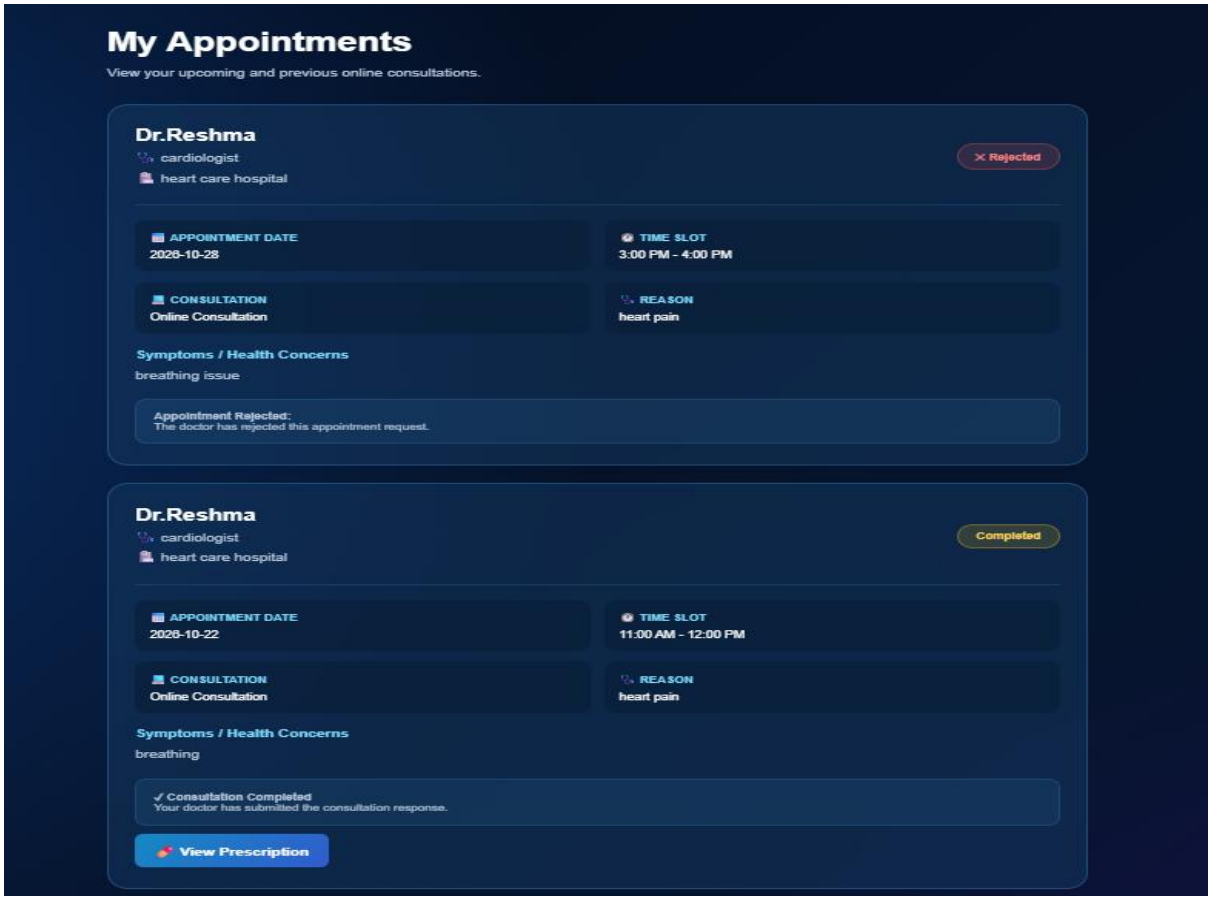
7.Doctor page: Displays doctor that is available in that hospital.



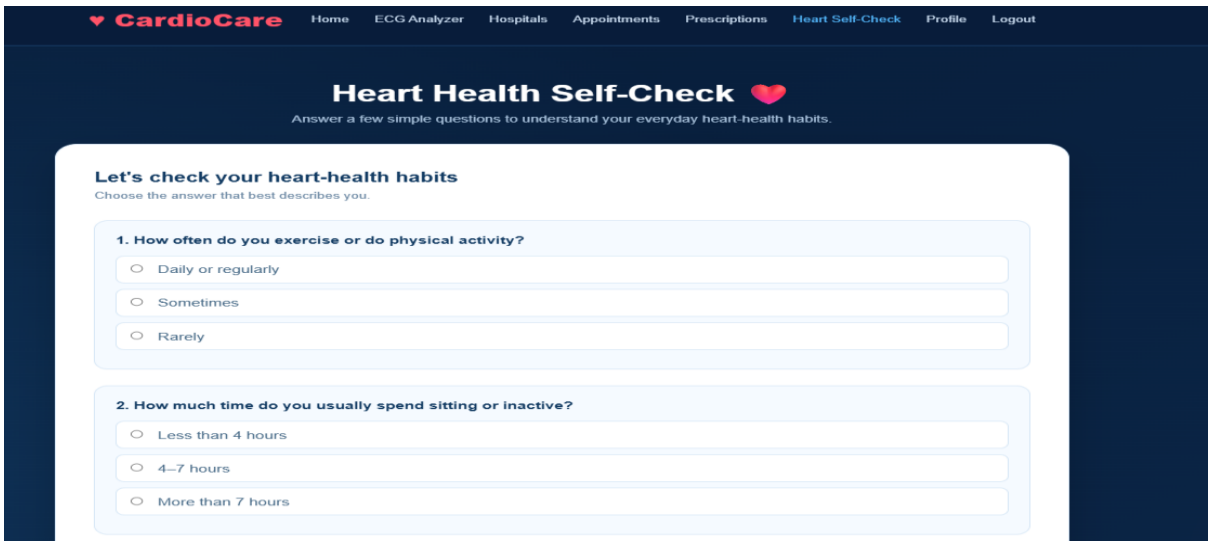
8. Book appointment page: User books appointment for the particular doctor



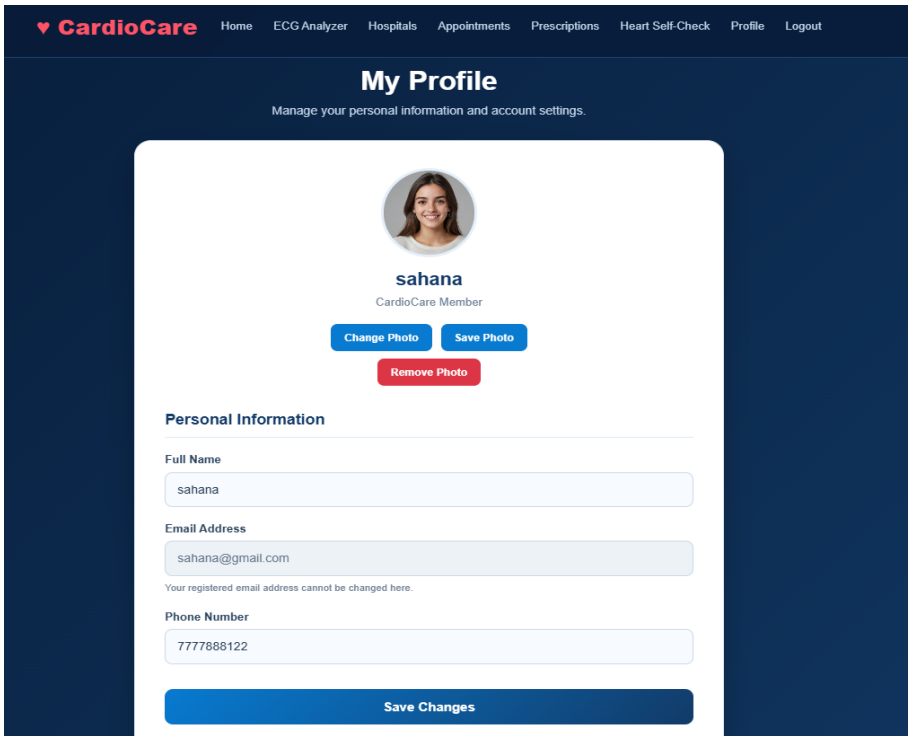
9.Appoinments page : User can view the appointments that he booked



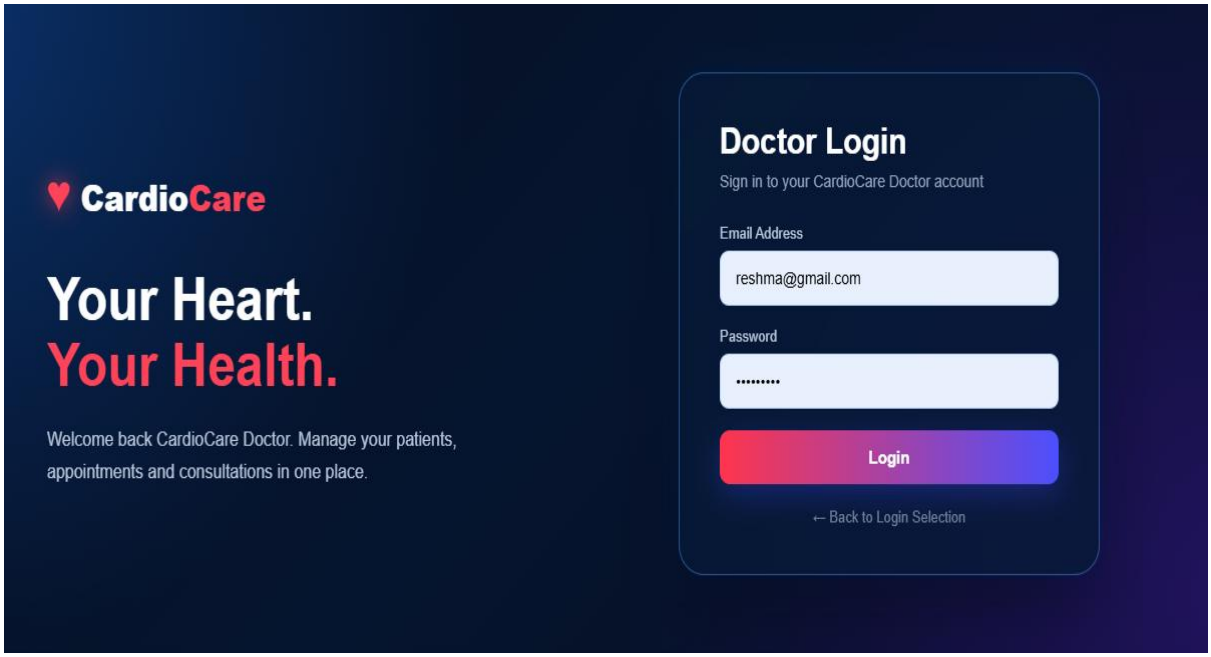
10.Heart self check page: User can answer the questions and understand the health score



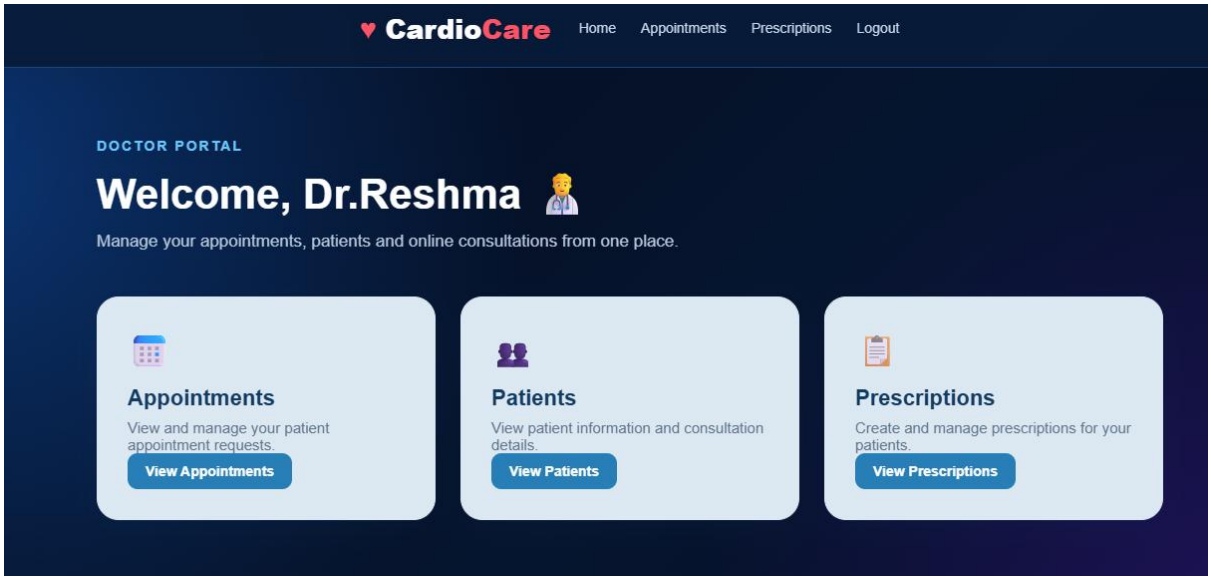
11.Profile page: This is the profile page of the user where he can add,delete photo and change password



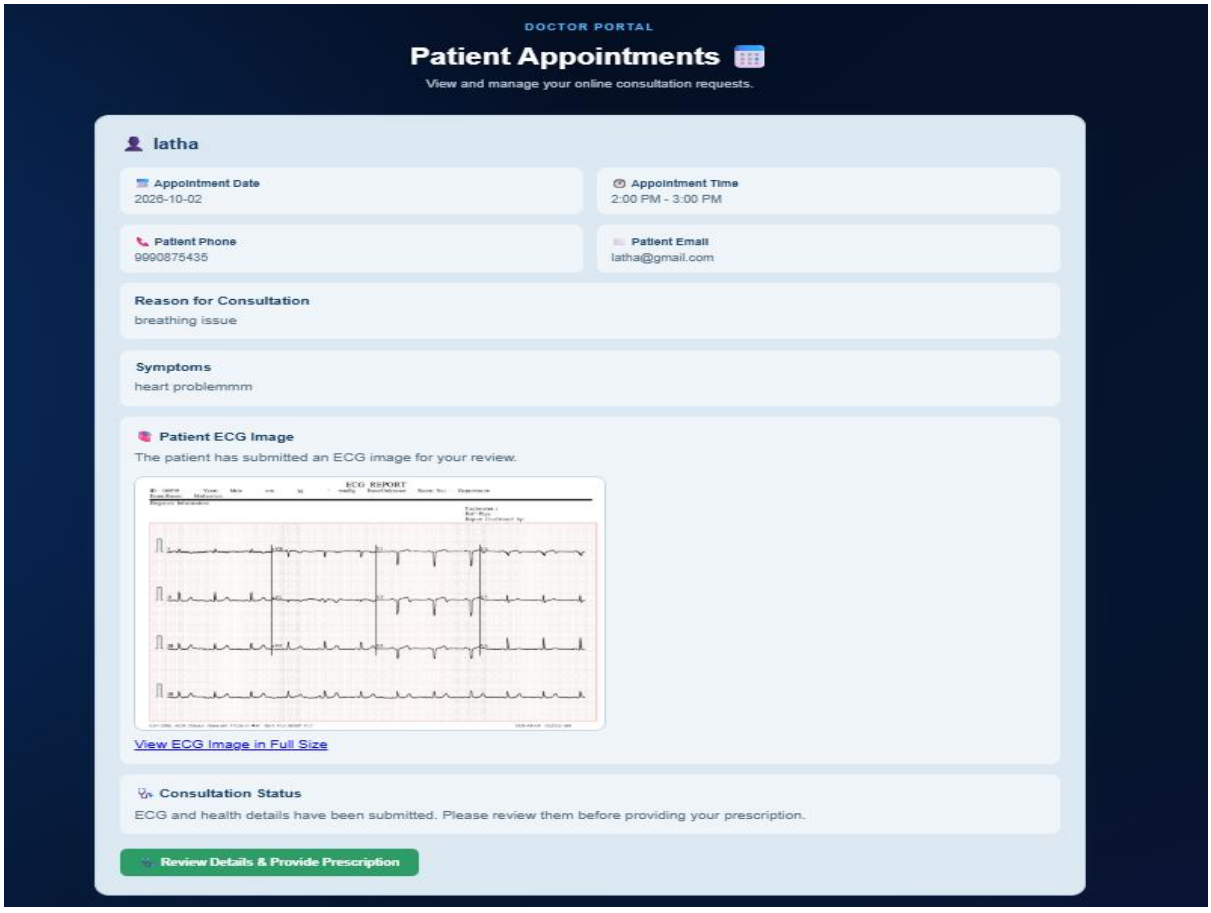
12.Doctor login: Doctor logins using login credentials**.**



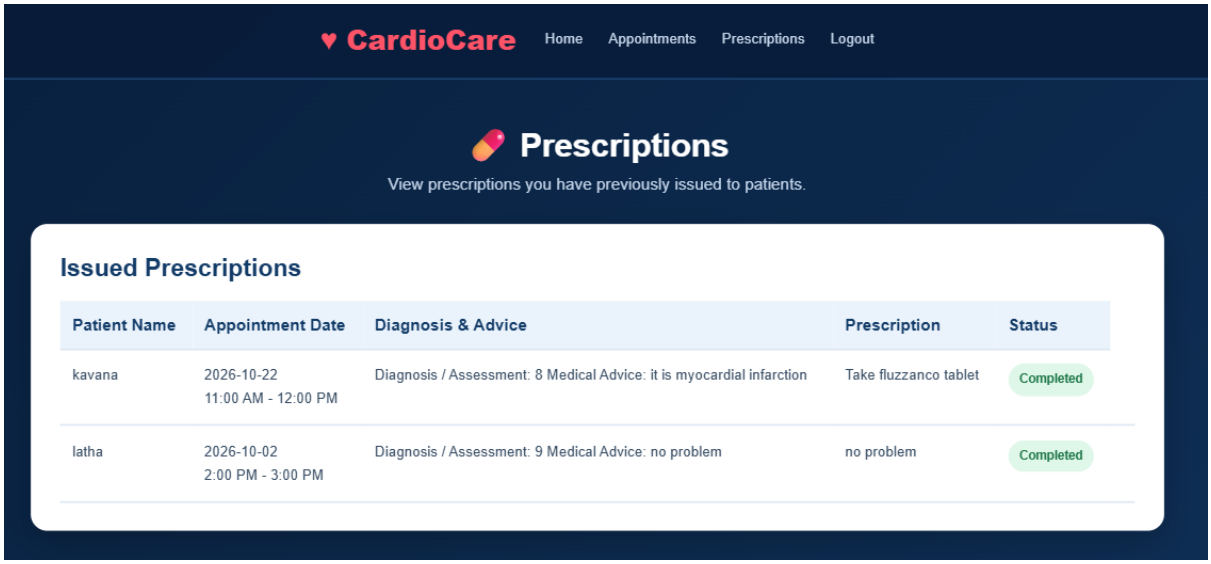
13. Doctor Dashboard: This the doctor dashboard displays appointment details.



14.Patient appointments: Doctor views appoinments made y user

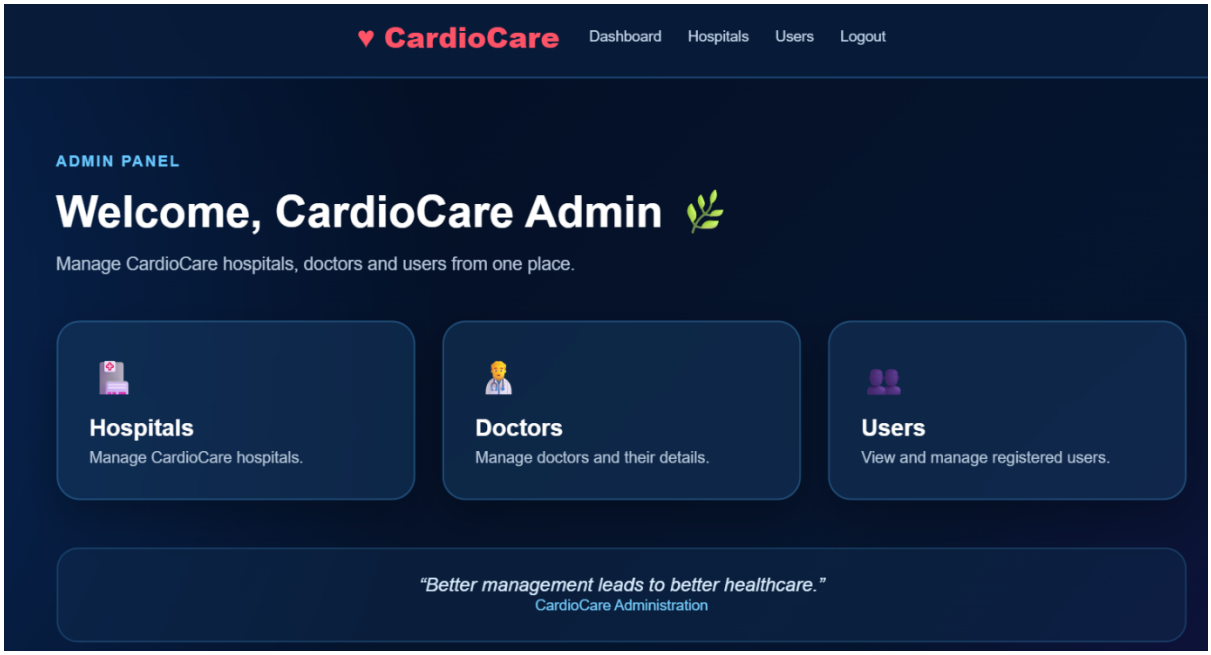


11.Presription page: Doctor views issued prescriptions.



12. Administrator Dashboard

The administrator can manage hospital and doctor information through the relevant management pages.



13.Add Hospital/doctor: Admin add new doctor and hospital



3. REFERENCES

1. A.J. Institute of Engineering & Technology, Department of MCA, *Software Design Specification (SDS) Format for Machine Learning Projects*.
2. PyTorch Documentation: https://pytorch.org/docs/
3. Torchvision Documentation: https://pytorch.org/vision/stable/
4. Flask Documentation: https://flask.palletsprojects.com/
5. SQLite Documentation: https://www.sqlite.org/docs.html
6. The original ECG dataset documentation. Add the dataset's actual source and licence or permission details.